

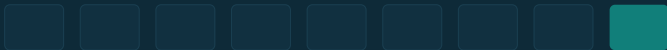


STREAM PROCESSING AVEC APACHE KAFKA

Réplication & durabilité

ISR, acks, rétention et compaction — et la tolérance aux pannes

ESGI · 5^e année · Data Engineering · 2025–2026



Objectifs de la séance

- **Comprendre comment la réplication protège les données**

Leader, followers, et l'ISR — l'ensemble des réplicas synchronisés.

- **Régler le compromis durabilité / latence**

Avec `acks` et `min.insync.replicas`.

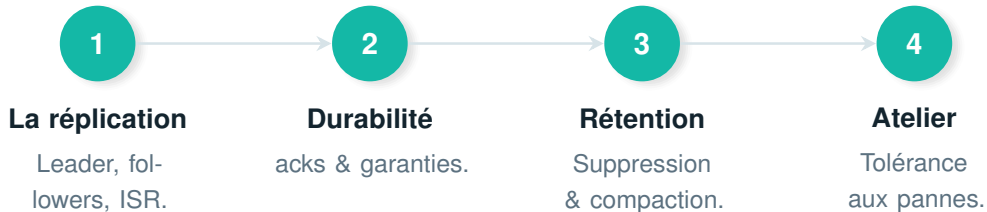
- **Maîtriser rétention et compaction des logs**

Combien de temps garde-t-on les données — et sous quelle forme.

- **Observer un cluster survivre à une panne**

On arrête un broker et on regarde l'ISR et les leaders réagir.

Au programme

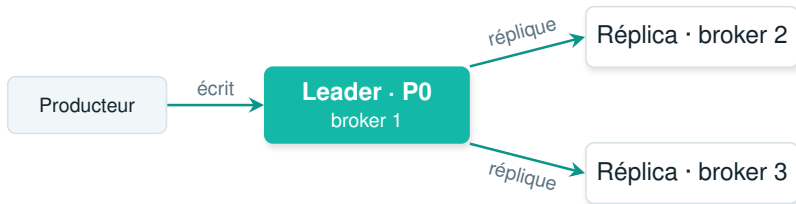


01

- PARTIE 1

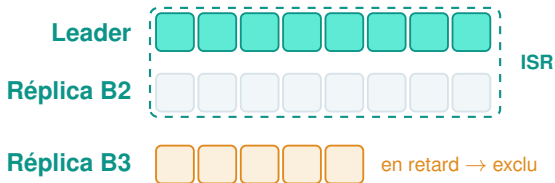
La réplication en détail

Comment une partition est répliquée



Le producteur n'écrit qu'au leader. Les réplicas (sur d'autres brokers) recopient son log en continu (fetch).

L'ISR : les réplicas synchronisés



L'ISR (In-Sync Replicas) = le leader et les réplicas à jour. Un réplica trop en retard (`replica.lag.time.max.ms`) en sort, et y revient une fois rattrapé.

Quand le leader tombe

Avant

B1 · leader

B2 · réplica

B3 · réplica

ISR : 1, 2, 3

Après la panne du broker 1

B1 · hors-service

B2 · leader

B3 · réplica

ISR : 2, 3

Un réplica de l'ISR (B2) est promu leader — automatiquement, et sans perte des écritures confirmées.

Et s'il ne reste aucun réplica à jour ?

■ Attendre (sûr)

La partition reste indisponible jusqu'au retour d'un réplica à jour. Aucune perte.

■ Unclean (risqué)

On élit un réplica en retard : la partition repart, mais des messages peuvent être perdus.

`unclean.leader.election.enable` arbitre disponibilité vs durabilité. Par défaut : `false` — on protège la donnée.

02

- PARTIE 2

Durabilité : acks & garanties

acks : le compromis durabilité / latence

■ acks = 0

N'attend aucun accusé. Le plus rapide, mais perte possible.

■ acks = 1

Attend le leader seul. Perte si le leader tombe avant réplication.

■ acks = all

Attend tous les réplicas de l'ISR. Le plus sûr — recommandé en prod.

De 0 à all : la latence augmente, la durabilité aussi. C'est un réglage par producteur.

LA VRAIE GARANTIE

`acks=all + min.insync.replicas`

$$RF = 3 \cdot \text{min.insync.replicas} = 2 \cdot \text{acks} = \text{all}$$

■ $ISR \geq 2$: écriture acceptée

Au moins 2 copies en phase. Le cluster tolère la perte d'un broker sans rien perdre.

■ $ISR < 2$: écriture refusée

Erreur `NOT_ENOUGH_REPLICAS` : Kafka préfère refuser plutôt que risquer une perte.

La garantie : une écriture confirmée en `acks=all` n'est jamais perdue tant que `min.insync.replicas` est respecté.

ELR : une élection de leader encore plus sûre

■ Le risque

Après plusieurs pannes, il fallait choisir entre indisponibilité et élection « unclean ».

■ L'idée (ELR)

Kafka mémorise des réplicas « éligibles » au-delà de l'ISR pour une élection toujours sûre.

■ En pratique

Présent dans Kafka 4.x : plus de robustesse face aux pannes, sans réglage particulier.

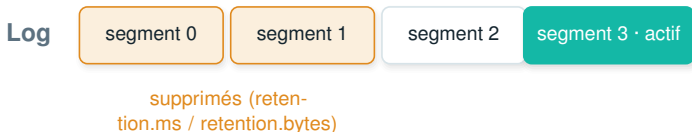
Eligible Leader Replicas (ELR) : moins de situations où l'on doit arbitrer entre disponibilité et perte de données.

03

- PARTIE 3

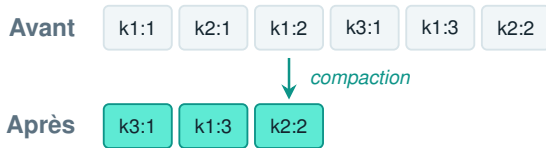
Rétention & compaction

Rétention : les logs ne grossissent pas à l'infini



`cleanup.policy=delete` (par défaut) : les segments au-delà de la durée ou de la taille configurée sont supprimés.

Compaction : la dernière valeur par clé



`cleanup.policy=compact` : on ne conserve que la dernière valeur de chaque clé. La clé devient un identifiant d'état.

delete ou compact ?

■ delete (par défaut)

Flux d'événements horodatés : commandes, clics, métriques, journaux, audit avec TTL.

■ compact

État courant par clé : changelogs d'état (KTable), topics de configuration, dernier profil connu.

On peut même combiner les deux (`compact`, `delete`) : garder la dernière valeur, puis purger au-delà d'un certain âge.

04

• PARTIE 4 · ATELIER

Tolérance aux pannes

Topic répliqué + acks=all

On reprend le cluster à 3 brokers et on vise la garantie de référence : RF 3, min.insync.replicas=2, acks=all.

bash

```
# topic durable : 3 copies, 2 minimum en phase
kafka-topics.sh --create --topic commandes \
  --partitions 3 --replication-factor 3 \
  --config min.insync.replicas=2 \
  --bootstrap-server localhost:9092

# producteur : acks=all
kafka-console-producer.sh --topic commandes \
  --producer-property acks=all \
  --bootstrap-server localhost:9092
```

Arrêter un broker et observer

- 1 Produire (acks=all)
- 2 `--describe` : noter l'ISR
- 3 Arrêter broker-2
- 4 `--describe` : l'ISR a changé
- 5 Reproduire : ça marche encore
- 6 Relancer : l'ISR se reconstitue

bash

```
# etat initial : Isr = 1,2,3
kafka-topics.sh --describe --topic commandes \
  --bootstrap-server localhost:9092
# couper un broker
docker compose stop broker-2
# re-decrire : leader bascule, Isr réduit
kafka-topics.sh --describe --topic commandes \
  --bootstrap-server localhost:9092
# relancer le broker
docker compose start broker-2
```

La bascule, avant / après

```
commandes -describe
```

```
# avant l'arret
```

```
Partition: 1  Leader: 2  Replicas: 2,3,1  Isr: 2,3,1
```

```
# apres l'arret de broker-2
```

```
Partition: 1  Leader: 3  Replicas: 2,3,1  Isr: 3,1
```

■ Ce qui change

Le leader passe de 2 à 3 ; le broker 2 quitte l'ISR.

■ Ce qui tient

$Isr = 2 \geq \text{min.insync.replicas}$: `acks=all` écrit toujours.

Récapitulatif

- ✓ Chaque partition est répliquée : un leader reçoit, des réplicas recopient.
- ✓ L'ISR est l'ensemble des réplicas à jour ; seuls eux peuvent devenir leader.
- ✓ `acks=all + min.insync.replicas` = la garantie de non-perte.
- ✓ `delete` purge par temps/taille ; `compact` garde la dernière valeur par clé.
- ✓ À la panne d'un broker, le leader bascule et le service continue.



LA SUITE — SÉANCE 4

Place au code : Java & l'API Kafka

Un échauffement Java, puis les API Producer et Consumer pour écrire et lire dans Kafka par programme — fini la ligne de commande.